CSE 2046/2246 Midterm

06. 05. 2023

2 Sessions (50 minutes each – including writing of the questions)

Cheat Sheets are not allowed!

“I am aware that any forms of cheating in this exam will result in a zero grade and a disciplinary investigation. I accept all rules and regulations regarding online exams. I give permission for the processing of my personal data as stated in the Clarification Text provided on the Faculty of Engineering website”

Name, Surname, ID, Signature

Write the above text at the top of your first page and sign it.

Take it easy!

Q1. (15 pts) **What is the time complexity of the following function? Give in Θ() notation**

```
void func(int n) {
  for (int i=n/2; i<=n; i++){
    for (int j=i; j>=1; j--) {
      for (int k=1; k<=n; k=k*3){
         printf{«Başarılar\n»}
      }
    }
  }
}
```

Q2. (20 pts) **How many lines does the following function print if it is called as `f(n,2)`? Write a recurrence relation and solve for even values of n.**
Notes: Do not forget that there are TWO inputs. Clearly give the base condition.

```
function f(n,m)
  if n <= m printline(«Kolay gelsin»);
  else
      for i=1 to m
            return f(n-2,m+2)
```

Q3. (20 pts) **A, B and C are three integer arrays of size m, n and t respectively. You are asked to print all the integers that exist in only one of A or B (but not in both), and does not exist in C. Describe an efficient pre-sorting based algorithm and give its time complexity.**
(Do not give pseudocode, give step-by-step description.)

Q4. (20 pts) Consider a text of 1000 elements with alternating 123, such as 123123123…

How many character comparisons will the **Boyer-Moore** algorithm make in searching for the pattern **332323** in this text? Show the bad-symbol, good suffix tables and how the algorithm is applied.

Q5. (30 pts) Consider a bit-string of length n. Your goal is to find the number of substrings that do not include 1. For example, if the input is 001000, the answer would be 9 (five 0, three 00, one 000).

(a – 20 pts) Describe a Divide and Conquer algorithm. Give the worst case time complexity of your algorithm.

(b – 10 pts) Describe another algorithm that is at least as good as (or better than) Divide and Conquer algorithm. Give its time complexity as well.

Do not give pseudocode! Step-by-step description please.

Q1. (15 pts) **What is the time complexity of the following function? Give in Θ() notation**

```
void func(int n) {
  for (int i=n/2; i<=n; i++){
    for (int j=i; j>=1; j--) {
      for (int k=1; k<=n; k=k*3){
         printf{«Başarılar\n»}
      }
    }
  }
}
```

$\lfloor \log_3 n + 1 \rfloor$ is more correct, but $\log n$ is also ok.

$$\sum_{i=\frac{n}{2}}^{n} \sum_{j=1}^{i} \log_3 n = \sum_{i=\frac{n}{2}}^{n} i \log_3 n.$$

$$= \left[\frac{n(n+1)}{2} - \frac{\left(\frac{n}{2}-1\right)\left(\frac{n}{2}\right)}{2} \right] \cdot \log_3 n$$

$$= \left[\frac{n^2}{2} + \frac{n}{2} - \frac{n^2}{8} + \frac{n}{4} \right] \log_3 n$$

$$= \left[\frac{3n^2}{8} + \frac{3n}{4} \right] \cdot \log_3 n \in \Theta(n^2 \log n)$$

Q2. (20 pts) **How many lines does the following function print if it is called as `f(n,2)`? Write a recurrence relation and solve for even values of n.**
Notes: Do not forget that there are TWO inputs. Clearly give the base condition.

```
function f(n,m)
  if n <= m printline(«Kolay gelsin»);
  else
      for i=1 to m
            return f(n-2,m+2)
```

$T(n,m) = 1$ if $n \le m$.

else:

$$T(n,2) = 2 \cdot T(n-2,4)$$

$$= 2.4 \cdot T(n-4,6)$$

$$= 2.4.6 \cdot T(n-6,8)$$

$$= 1.2.3 \ldots \left(\frac{n}{4}\right) \cdot \underbrace{2.2.2 \ldots 2}_{\frac{n}{2} \text{ times}}$$

if $n = 4k \Rightarrow$

$$= 2.4.6 \ldots \left(\frac{n}{2}\right) \underbrace{T\left(n - \frac{n}{2}, 2 + \frac{n}{2}\right)}_{1}$$

$$= 2^{\frac{n}{4}} \cdot \left(\frac{n}{4}\right)!$$

if $n = 4k+2 \Rightarrow$

$$= 2.4.6 \ldots \left(\frac{n-2}{2}\right) T\left(\underbrace{n - \frac{n-2}{2}}_{\frac{n+2}{2}}, \underbrace{2 + \frac{n-2}{2}}_{\frac{n+2}{2}}\right)$$

$$= 2^{\frac{n-2}{4}} \cdot \left(\frac{n-2}{4}\right)!$$

This is the exact solution. But similar solutions may get full grade.

Q3. (20 pts) **A, B and C are three integer arrays of size m, n and t respectively. You are asked to print all the integers that exist in only one of A or B (but not in both), and does not exist in C. Describe an efficient pre-sorting based algorithm and give its time complexity.**
(Do not give pseudocode, give step-by-step description.)

1st alternative:

Sort B using Mergesort. $\Theta(n \log n)$

Search all elements of A in sorted B. If exist erase that element from A. $O(m \log n)$

Binary search.

Sort C using Mergesort. $\Theta(t \log t)$

Search all elements in A & B in sorted C. $O((m+n)\log t)$

If not exist, print. Time complexity = $O((m+n)\log n + (m+n+t)\log t)$

Note: If you don't want to remove any element from A, you may use a temporary array D and write the elements that do not exist in B to this array.

2nd alternative:

Sort A using Mergesort. $\Theta(m \log m)$

Sort B using Mergesort $\Theta(n \log n)$

Sort C using Mergesort $\Theta(t \log t)$

Search all elements of A in sorted B using Binary Search. $O(m \log n)$

If not exists, search in sorted C using Binary Search. $O(m \log t)$

If not exists, print.

Search all elements of B in sorted A using Binary Search. $O(n \log m)$

If not exists, search in sorted C using Binary Search. $O(n \log t)$

If not exists, print.

Time complexity: $O((m+n)\log m + (m+n)\log n + (m+n+t)\log t)$.

3rd alternative:

Merge sort C.

Merge A & B, and sort the merged list $\Theta((m+n)\log(m+n))$

Discard the adjacent repeating elements, and search the other elements in sorted C. If not exist, print. $O(m+n)+O((m+n)\log t)$

Time complexity: $\Theta(m+n)\log(m+n) + O((m+n)\log t)$.

Note: This alternative will work only if A & B do not have repetitions.

Q4. (20 pts) Consider a text of 1000 elements with alternating 123, such as 123123123…

How many character comparisons will the Boyer-Moore algorithm make in searching for the pattern 332323 in this text? Show the bad-symbol, good suffix tables and how the algorithm is applied.

Bad symbol table:

1	2	3	others
6	1	2	6

Good suffix:

k		d_2
1	332323	4
2	332323	5
3	332323	2
4	332323	5
5	332323	5

1 2 3 1 2 3 1 2 3 1 2 3 . . .

3 3 2 3 2 3

3 3 2 3 2 3

3 3 2 3 2 3

3 3 2 3 2 3

$d_1 = 6-2=4$

$d_2 = 5$ ✓

shift by 5.

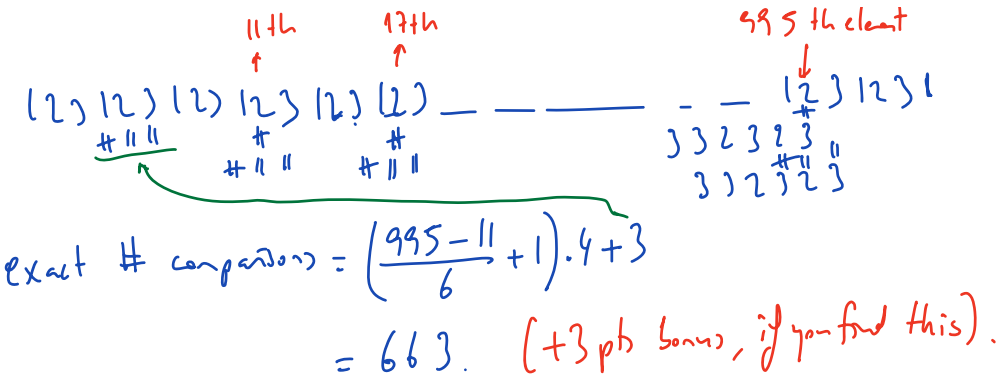
Shift by 1.

Repeatedly: 4 comparisons 6 shift.

approximate # comparisons: $1000 \cdot \frac{4}{6}$

≈ 667.

This answer would be ok.



Q5. (30 pts) **Consider a bit-string of length n. Your goal is to find the number of substrings that do not include 1. For example, if the input is 001000, the answer would be 9 (five 0, three 00, one 000).**

(a – 20 pts) **Describe a <u>Divide and Conquer</u> algorithm. Give the worst case time complexity of your algorithm.**

if length of the bit string is 1, return 1 if its value is 0, return 0 otherwise.

Else:

Divide the bitstring into two halves from the middle.

Recursively find the # substrings that do not include 1 in the left substring.: N_L

" " " " " " " " " right " : N_R

From the middle, count # zeros towards left until 1 is encountered: z_L

count # zeros " right " 1 " " : z_R

Return $N_L + N_R + z_L \cdot z_R$ → This is the number of substrings (that do not include 1) and some part is in left, some part is in right half.

Example: 1 0 0 0 | 0 0 1

3 2

0 | 0
0 | 0 0
0 0 | 0
0 0 | 0 0
0 0 0 | 0
0 0 0 | 0 0

There are 3×2 such strings.

$T(n) = 2T\left(\frac{n}{2}\right) + f(n)$ where $f(n) \in O(n)$.

$T(n) \in O(n \log n)$.

(b – 10 pts) Describe another algorithm that is at least as good as (or better than) Divide and Conquer algorithm. Give its time complexity as well.

Do not give pseudocode! Step-by-step description please.

Step 1: Set counter to zero. Start from beginning of the bitstring.

Step 2: Count number of zeros until encountering 1. Let's say it is n_z.

step 3: counter = counter + $\frac{n_z(n_z+1)}{2}$

Step 4: Proceed until encountering $\emptyset$. If there is no $\emptyset$ return counter, otherwise go to step 2.

Time complexity = $\Theta(n)$.

Example: Consider 001000

00 → 1+2 = 3 substrings

000 → 1+2+3 = $\frac{3.4}{2}$ substrings.